Filecoin Sealing 최적화 구현체 분석

개요

SupraSeal은 Filecoin 스토리지 제공자를 위한 고도로 최적화된 sealing 프리미티브 컬렉션입니다. 기존 rust-fil-proofs 대비 처리량 대비 비용(throughput per dollar)을 극적으로 개선하여 높은 처리량이 필요한 스토리지 제공자들을 위해 설계되었습니다.

Filecoin Sealing 프로세스 개요

Filecoin sealing은 다음 단계로 구성됩니다:

1. **Add Piece**: 데이터 조각들을 연결하여 섹터(예: 32GB)를 채우는 과정
2. **Pre-commit 1 (PC1)**: Stacked Depth Robust Graph 생성
3. **Pre-commit 2 (PC2)**: 그래프 컬럼과 복제본으로 구성된 두 개의 Merkle 트리 생성
4. **Wait Seed**: 체인에서 랜덤성(시드) 획득을 위한 150 에포크(~75분) 대기
5. **Commit 1 (C1)**: 시드를 사용한 노드 챌린지 생성 및 포함 증명 구축
6. **Commit 2 (C2)**: C1의 포함 증명을 사용한 zkSNARK 증명 생성

1. Pre-Commit 1 (PC1) 최적화

1.1 핵심 최적화 전략

메모리를 NVMe로 대체

- **기존 문제**: 기존 구현은 1-2TB RAM을 사용하여 15-30개 섹터를 병렬 처리
- **해결책**: 대부분의 데이터를 NVMe에 저장하고 작은 인메모리 캐시만 유지

- **효과**: 32코어 시스템에서 64섹터 처리 시 256GB 메모리만 필요 (기존 대비 75% 감소)

사용자 공간 I/O (SPDK 활용)

- **기존 문제**: 커널 시스템 콜과 파일시스템 오버헤드
- **해결책**: SPDK를 통한 직접적인 NVMe 제어
- **효과**: I/O 지연시간 감소 및 CPU 오버헤드 제거

코어 활용률 증대

- **기존 문제**: 섹터당 4개 코어 사용 (코어 활용률 저하)
- **해결책**: 코어당 4개 섹터 처리 (멀티버퍼 접근법)
- **효과**: 필요 코어 수 16배 감소

1.2 SHA-256 최적화

SHA Extensions 활용

```
각 노드당 필요한 연산:
- 20 블록 × 64 라운드 ÷ 2 라운드/명령어 = 640회 sha256rnds2 실행
- 640 × 4 사이클 = 2,560 사이클/노드
- 32GB 그래프: 11 레이어 × 2^30 노드 × 2,560 사이클 = 30조 사이클
```

멀티버퍼 구현

- **x2 구현**: 레지스터 압박 없이 2배 처리량 달성
- **x4 구현**: 하이퍼스레딩으로 4배 처리량 (코어당 2개 스레드 × 2버퍼)
- **성능**: 4200 사이클로 4버퍼 처리 (단일 버퍼 대비 2.4배 효율)

1.3 데이터 레이아웃 최적화

인터리브드 섹터 레이아웃

```
기존: [섹터0 노드1-N] [섹터1 노드1-N] ...

최적화: [모든섹터 노드1] [모든섹터 노드2] ...
```

페이지 활용률 개선

- **기존**: 4KB 읽기에서 32B만 사용 (0.78% 활용률)
- **최적화**: 4KB 읽기로 64개 섹터의 동일 노드 획득 (100% 활용률)
- **효과**: 디스크 I/O 요구사항 64배 감소

1.4 멀티 디스크 아키텍처

용량 요구사항

- 64섹터 × 32GB × 11레이어 = 22TB 저장공간 필요

IOPS 요구사항

- 노드당 14개 부모 중 10개는 랜덤 읽기 필요
- 3.5시간 내 완료를 위해 약 1,000만 IOPS 필요
- 드라이브당 100만 IOPS × 12개 드라이브로 달성

스트라이핑 전략

```
페이지 0 → 디스크 0

페이지 1 → 디스크 1

페이지 2 → 디스크 2

페이지 3 → 디스크 3
```

```
페이지 4 → 디스크 0 (순환)
```

1.5 소프트웨어 아키텍처

스레드 구조

- **Orchestrator**: 전체 데이터 플로우 관리 (전용 코어)
- **Parent Reader**: NVMe에서 부모 노드 읽기 (전용 코어)
- **Node Writer**: 해시된 노드를 NVMe에 쓰기 (전용 코어)
- **Coordinators**: CCX당 1개, L3 캐시에 데이터 배치
- **Hashers**: 실제 SHA-256 해싱 수행 (코어당 2개 스레드)

버퍼 관리

- **Parent Buffer**: 디스크에서 읽은 부모 노드 데이터
- **Node Buffer**: 해시된 노드 저장 및 인메모리 캐시 역할
- **Parent Pointers**: 부모 데이터 포인터 (버퍼 또는 캐시)
- **Sync Buffers**: 동기화 데이터

메모리 최적화

- **Huge Pages**: 1GB 페이지로 TLB 미스 감소
- **Pinned Memory**: DMA를 위한 메모리 고정
- **NUMA 인식**: CCX 구조에 맞춘 스레드 배치

2. Pre-Commit 2 (PC2) 최적화

2.1 병렬 Tree C 생성

Poseidon 해시 최적화

- **GPU 가속**: CUDA를 통한 대규모 병렬 처리
- **배치 처리**: 여러 섹터의 컬럼을 동시에 처리
- **스트림 활용**: 64개 스트림으로 GPU 활용률 극대화

메모리 접근 패턴 최적화

```
// 컬럼별 데이터 접근 최적화
for (size_t j = 0; j < C::GetNumLayers(); j++) {
for (size_t k = 0; k < C::GetNumNodes(); k++) {
fr_t val = data[j * C::GetNumNodes() * C::PARALLEL_SECTORS +
k * C::PARALLEL_SECTORS + i];
tree_c_input[k * C::GetNumLayers() + j] = val;
}
}
```

2.2 Tree R 생성 최적화

CC 섹터 최적화

- **병렬 처리**: 모든 섹터의 Tree R을 동시에 생성
- **마지막 레이어 활용**: NVMe의 인터리브된 마지막 레이어 직접 사용

비CC 섹터 처리

- **로컬 처리**: 데이터가 로컬에 있는 경우 단일 섹터 처리
- **원격 처리**: 데이터가 원격에 있는 경우 원격에서 Tree R 생성 후 루트만 전송

2.3 GPU 최적화

CUDA 아키텍처 활용

```
// build.rs에서 다중 아키텍처 지원
nvcc.flag("-arch=sm_80");
nvcc.flag("-gencode").flag("arch=compute_70,code=sm_70");
```

메모리 계층 최적화

- **공유 메모리**: 스레드 블록 내 데이터 공유
- **텍스처 메모리**: 읽기 전용 데이터 캐싱
- **상수 메모리**: 라운드 상수 저장

3. Commit 1 (C1) 최적화

3.1 챌린지 생성 최적화

단일 섹터 처리

- **이유**: 챌린지의 랜덤성으로 인해 병렬화 효과 제한
- **전략**: 섹터별 순차 처리로 복잡성 감소

포함 증명 생성

```
// Tree D와 Tree R 증명을 동시에 생성
int c1(size_t block_offset, size_t num_sectors, size_t sector_slot,
const uint8_t* replica_id, const uint8_t* seed,
const uint8_t* ticket, const char* cache_path,
```

```
const char* parents_filename);
```

3.2 메모리 접근 최적화

캐시 친화적 접근

- **지역성 활용**: 연속된 노드들의 증명을 함께 생성
- **프리페칭**: 다음 챌린지에 필요한 데이터 미리 로드

I/O 최적화

- **배치 읽기**: 여러 증명에 필요한 데이터를 한 번에 읽기
- **비동기 I/O**: 증명 생성과 데이터 로딩 병렬화

4. Commit 2 (C2) 최적화

4.1 Groth16 증명 최적화

GPU 가속 MSM (Multi-Scalar Multiplication)

- **병렬 처리**: 수천 개의 스칼라 곱셈을 GPU에서 병렬 처리
- **메모리 대역폭 최적화**: 효율적인 메모리 접근 패턴

NTT (Number Theoretic Transform) 최적화

- **FFT 가속**: GPU의 병렬성을 활용한 고속 FFT
- **인플레이스 연산**: 메모리 사용량 최소화

4.2 회로 최적화

배치 처리

```
// bellperson과의 통합을 통한 배치 증명 생성

// cuda-supraseal 기능을 통한 GPU 가속
```

메모리 관리

- **스트리밍**: 대용량 데이터의 스트리밍 처리
- **가비지 컬렉션**: 불필요한 중간 결과 즉시 해제

5. 시스템 레벨 최적화

5.1 하드웨어 최적화

참조 플랫폼

- **CPU**: Threadripper PRO 5995WX (64코어)
- **메모리**: 512GB DDR4
- **스토리지**: 16 × Samsung 7.68TB U.2 NVMe
- **GPU**: Nvidia RTX 4090
- **네트워크**: 4 × Supermicro NVMe HBA

메모리 구성

```
# 1GB Huge Pages 설정

GRUB_CMDLINE_LINUX="default_hugepagesz=1G hugepagesz=1G hugepages=36"
```

5.2 파일시스템 최적화

F2FS 사용

- **플래시 최적화**: SSD에 특화된 파일시스템
- **Lazy Time**: 메타데이터 업데이트 지연으로 성능 향상

디스크 분리

- **PC1**: SPDK 제어 디스크 (읽기 집약적)
- **PC2**: 파일시스템 디스크 (쓰기 집약적)
- **부모 캐시**: 별도 디스크로 I/O 경합 방지

5.3 네트워크 최적화

PCIe 대역폭 관리

- **컨트롤러 분산**: NVMe 컨트롤러를 여러 PCIe 슬롯에 분산
- **NUMA 인식**: 메모리와 CPU의 지역성 고려

인터커넥트 최적화

- **직접 연결**: CPU-GPU 간 직접 PCIe 연결
- **대역폭 균형**: 각 구성요소 간 대역폭 균형 유지

6. 성능 결과

6.1 처리량 개선

- **PC1**: 64섹터를 3시간 30분에 처리 (이론적 최적값 3시간 26분에 근접)
- **메모리 사용량**: 256GB (기존 1-2TB 대비 75% 감소)
- **코어 효율성**: 코어당 4섹터 처리 (기존 섹터당 4코어 대비 16배 개선)

6.2 비용 효율성

- **하드웨어 비용**: 메모리 요구사항 감소로 시스템 비용 대폭 절감
- **전력 효율성**: 높은 코어 활용률로 전력 대비 성능 향상
- **확장성**: 병렬 처리 능력으로 대규모 운영 가능

6.3 안정성

- **에러 처리**: 각 단계별 체크포인트와 복구 메커니즘
- **모니터링**: 실시간 성능 통계 및 병목 지점 식별
- **테스트**: 광범위한 단위 테스트 및 통합 테스트

결론

SupraSeal은 Filecoin sealing 프로세스의 모든 단계에서 혁신적인 최적화를 달성했습니다:

1. **PC1**: 메모리-NVMe 하이브리드 아키텍처와 멀티버퍼 SHA-256으로 처리량 16배 향상
2. **PC2**: GPU 가속 Poseidon 해시와 병렬 트리 생성으로 대폭적인 성능 개선
3. **C1**: 효율적인 증명 생성과 캐시 최적화로 지연시간 최소화
4. **C2**: GPU 가속 Groth16 증명으로 zkSNARK 생성 시간 단축

이러한 최적화를 통해 기존 대비 처리량 대비 비용을 극적으로 개선하여, Filecoin 스토리지 제공자들이 더 경쟁력 있는 서비스를 제공할 수 있게 되었습니다.